



Programming Fundamentals

Variables, Data Types, and Arithmetic Operators

Programming Fundamentals Team

SETU

2026-06-04



Outline

1. Variables

2. Data Types

3. Arithmetic Operators



Outline

1. Variables

2. Data Types

3. Arithmetic Operators



1. Variables

In Programming, variables:

- are created (defined) in your programs.



1. Variables

In Programming, variables:

- are created (defined) in your programs.
- are used to store data (value can change over time).



1. Variables

In Programming, variables:

- are created (defined) in your programs.
- are used to store data (value can change over time).
- have a data type.



1. Variables

In Programming, variables:

- are created (defined) in your programs.
- are used to store data (value can change over time).
- have a data type.
- have a name.



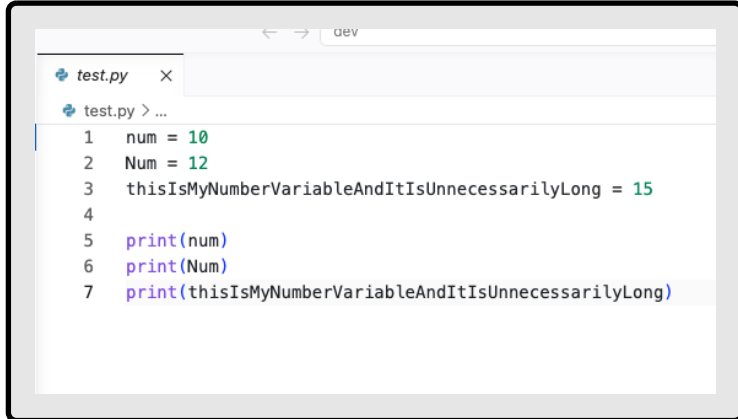
1. Variables

In Programming, variables:

- are created (defined) in your programs.
- are used to store data (value can change over time).
- have a data type.
- have a name.
- are a VERY important programming concept



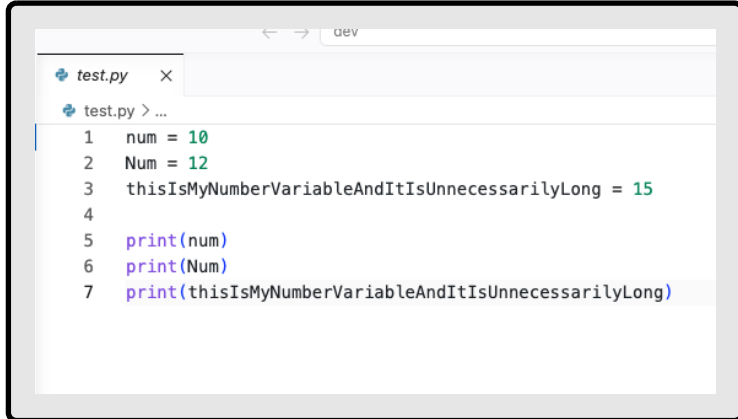
1.1 Variable Names



```
test.py  X
test.py > ...
1 num = 10
2 Num = 12
3 thisIsMyNumberVariableAndItIsUnnecessarilyLong = 15
4
5 print(num)
6 print(Num)
7 print(thisIsMyNumberVariableAndItIsUnnecessarilyLong)
```



1.1 Variable Names



```
test.py  X
test.py > ...
1 num = 10
2 Num = 12
3 thisIsMyNumberVariableAndItIsUnnecessarilyLong = 15
4
5 print(num)
6 print(Num)
7 print(thisIsMyNumberVariableAndItIsUnnecessarilyLong)
```

...should be short and meaningful. num is better than n.



1.1 Variable Names

test2.py > ...

```
1  numberOne = 10
```

```
2  num10 = 12
```

```
3  first_Number = 15
```



1.1 Variable Names

test2.py > ...

```
1  numberOne = 10
```

```
2  num10 = 12
```

```
3  first_Number = 15
```

can contain

- letters(uppercase and lowercase),
- digits(0-9), and
- underscores (_).



1.1 Variable Names

test3.py > ...

```
1  firstNumber = 10
2  secondNumber = 12
3  numberOfItems = 15
```



1.1 Variable Names

test3.py > ...

```
1  firstNumber = 10
2  secondNumber = 12
3  numberOfItems = 15
```

- must start with a letter or an underscore.
- should use camelCase when combining multiple words.



1.1 Variable Names

```
>>> num = 10
>>> print(Num)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    print(Num)
          ^^^
NameError: name 'Num' is not defined. Did you mean: 'num'?
```

Note : This uses the ‘run from Python REPL’ feature in Visual Studio Code, which allows you to execute Python code directly from the editor and see the output in the integrated terminal.



1.1 Variable Names

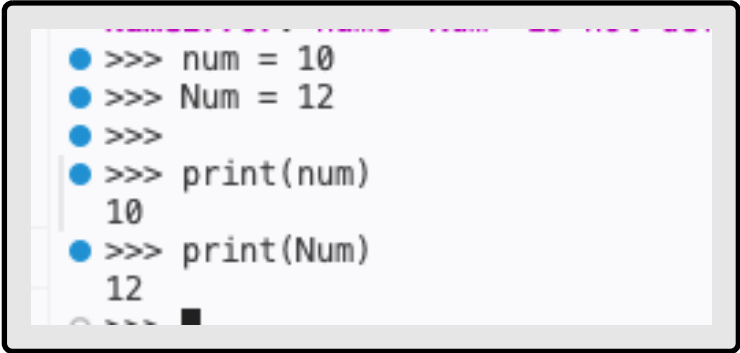
```
>>> num = 10
>>> print(Num)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    print(Num)
          ^^^
NameError: name 'Num' is not defined. Did you mean: 'num'?
```

- are CASE sensitive.

Note : This uses the 'run from Python REPL' feature in Visual Studio Code, which allows you to execute Python code directly from the editor and see the output in the integrated terminal.



1.1 Variable Names



```
>>> num = 10
>>> Num = 12
>>>
>>> print(num)
10
>>> print(Num)
12
>>>
```

Note : Again, this uses the 'run from Python REPL' feature in Visual Studio Code, which allows you to execute Python code directly from the editor and see the output in the integrated terminal.



1.1 Variable Names

```
>>> num = 10
>>> Num = 12
>>>
>>> print(num)
10
>>> print(Num)
12
```

- are CASE sensitive.
- num and NUM are different variables.

Note : Again, this uses the 'run from Python REPL' feature in Visual Studio Code, which allows you to execute Python code directly from the editor and see the output in the integrated terminal.



1.1 Variable Names

```
test4.py > firstNumber
1 firstNumber = 10
2 if = 20

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
Open file in editor (cmd + click) :rs/maireadmeagher/Documents/Gi
/Users/maireadmeagher/.zshrc:25: unmatched "
maireadmeagher@Mac dev % /opt/homebrew/bin/python3 /Users/ma
maireadmeagher/Documents/GitRepos/Python-PF/dev/test4.py
File "/Users/maireadmeagher/Documents/GitRepos/Python-PF/d
ev/test4.py", line 2
    if = 20
      ^
SyntaxError: invalid syntax
maireadmeagher@Mac dev %
```

Note : This uses the ‘right-click and run’ feature in Visual Studio Code, which allows you to execute Python code directly from the editor and see the output in the integrated terminal.



1.1 Variable Names

```
test4.py > firstNumber
1 firstNumber = 10
2 if = 20
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS +

Open file in editor (cmd + click) :rs/maireadmeagher/Documents/Gi

```
/Users/maireadmeagher/.zshrc:25: unmatched "
maireadmeagher@Mac dev % /opt/homebrew/bin/python3 /Users/ma
maireadmeagher/Documents/GitRepos/Python-PF/dev/test4.py
File "/Users/maireadmeagher/Documents/GitRepos/Python-PF/d
ev/test4.py", line 2
  if = 20
    ^
SyntaxError: invalid syntax
maireadmeagher@Mac dev %
```

- cannot be a keyword/reserved word

Note : This uses the 'right-click and run' feature in Visual Studio Code, which allows you to execute Python code directly from the editor and see the output in the integrated terminal.



1.1 Variable Names

```
test4.py > ...
1 firstNumber = 10
2 another number = 15
```

2

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS +

maireadmeagher@Mac dev %
maireadmeagher@Mac dev % /opt/homebrew/bin/python3 /Users/maireadmeagher/Documents/GitRepos/Python-PF/dev/test4.py
File "/Users/maireadmeagher/Documents/GitRepos/Python-PF/dev/test4.py", line 2
another number
~~~~~  
SyntaxError: invalid syntax. Did you mean 'not'?  
maireadmeagher@Mac dev %

**Note :** This uses the ‘right-click and run’ feature in Visual Studio Code, which allows you to execute Python code directly from the editor and see the output in the integrated terminal.



# 1.1 Variable Names

```
test4.py > ...
1 firstNumber = 10
2 another number = 15

SyntaxError: invalid syntax. Did you mean 'not'?
maireadmeagher@Mac dev %
```

- cannot contain spaces or special characters (except underscores)

**Note :** This uses the 'right-click and run' feature in Visual Studio Code, which allows you to execute Python code directly from the editor and see the output in the integrated terminal.



# 1.1 Variable Names

```
test5.py > ...  
1 firstNumber = 14  
2 _number = 12  
3 9number = 34  
  
PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS  
maireadmeagher@Mac dev % /opt/homebrew/bin/python3 /Users/ma  
ireadmeagher/Documents/GitRepos/Python-PF/dev/test5.py  
File "/Users/maireadmeagher/Documents/GitRepos/Python-PF/d  
ev/test5.py", line 3  
  9number = 34  
  ^  
SyntaxError: invalid decimal literal
```

**Note :** This uses the ‘right-click and run’ feature in Visual Studio Code, which allows you to execute Python code directly from the editor and see the output in the integrated terminal.



# 1.1 Variable Names

The screenshot shows a Visual Studio Code editor window with a file named `test5.py`. The code contains three lines:

```
1 firstNumber = 14
2 _number = 12
3 9number = 34
```

The third line, `9number = 34`, has a red squiggly line under the `9`, indicating a syntax error. Below the code editor, the `TERMINAL` tab is active, showing the following output:

```
maireadmeagher@Mac dev % /opt/homebrew/bin/python3 /Users/maireadmeagher/Documents/GitRepos/Python-PF/dev/test5.py
File "/Users/maireadmeagher/Documents/GitRepos/Python-PF/dev/test5.py", line 3
  9number = 34
  ^
SyntaxError: invalid decimal literal
```

- must start with a letter or an underscore

**Note** : This uses the 'right-click and run' feature in Visual Studio Code, which allows you to execute Python code directly from the editor and see the output in the integrated terminal.





## 1.2 Variable Assignment

- In Python, you create a variable by assigning a value to it using the assignment operator `=`. The syntax for variable assignment is:

```
variable_name = value
```



## 1.2 Variable Assignment

- In Python, you create a variable by assigning a value to it using the assignment operator `=`. The syntax for variable assignment is:

```
variable_name = value
```

For example, to create a variable named `num` and assign it the value `10`, you would write:

```
num = 10
```



## 1.2 Variable Assignment

- In Python, you create a variable by assigning a value to it using the assignment operator `=`. The syntax for variable assignment is:

```
variable_name = value
```

For example, to create a variable named `num` and assign it the value `10`, you would write:

```
num = 10
```

- A variable stores a single value, so any previous value is lost.



## 1.2 Variable Assignment

- In Python, you create a variable by assigning a value to it using the assignment operator `=`. The syntax for variable assignment is:

```
variable_name = value
```

For example, to create a variable named `num` and assign it the value `10`, you would write:

```
num = 10
```

- A variable stores a single value, so any previous value is lost.
- Assignment statements work by taking the value of what appears on the right-hand side of the operator and copying that value into a variable on the left-hand side.



# Outline

1. Variables

**2. Data Types**

3. Arithmetic Operators



## 2. Data Types

- A data type is a classification that specifies which type of value a variable can hold and what operations can be performed on it.



## 2.1 Variables and Data Types

test6.py > ...

```
1  firstNumber = 10
2
3  # name of variable is firstNumber
4  # value of variable is 10
5  # type of variable is int (integer)
6  # memory is managed by Python
```



## 2.1 Variables and Data Types

```
test6.py > ...  
1  firstNumber = 10  
2  
3  # name of variable is firstNumber  
4  # value of variable is 10  
5  # type of variable is int (integer)  
6  # memory is managed by Python
```

Each python variable has:

- name of variable





## 2.1 Variables and Data Types

```
test6.py > ...  
1  firstNumber = 10  
2  
3  # name of variable is firstNumber  
4  # value of variable is 10  
5  # type of variable is int (integer)  
6  # memory is managed by Python
```

Each python variable has:

- name of variable
- value of variable



## 2.1 Variables and Data Types

```
test6.py > ...  
1  firstNumber = 10  
2  
3  # name of variable is firstNumber  
4  # value of variable is 10  
5  # type of variable is int (integer)  
6  # memory is managed by Python
```

Each python variable has:

- name of variable
- value of variable
- type of variable



## 2.1 Variables and Data Types

test6.py > ...

```
1  firstNumber = 10
2
3  # name of variable is firstNumber
4  # value of variable is 10
5  # type of variable is int (integer)
6  # memory is managed by Python
```

Each python variable has:

- name of variable
- value of variable
- type of variable
- address (in memory) of variable.



## 2.2 Data Types in Python

The data type defines the kinds of values (data) that can be stored in the variable e.g.

- - 456
- 2
- 45.7897
- I Love Programming
- S
- True
- A, B, C, D, E, F, G



## 2.2 Data Types in Python

The data type defines the kinds of values (data) that can be stored in the variable e.g.

- - 456
- 2
- 45.7897
- I Love Programming
- S
- True
- A, B, C, D, E, F, G

The data type determines the operations that may be performed on it.



## 2.2 Data Types in Python

Python has 4 Primitive Data Types:



## 2.2 Data Types in Python

Python has 4 Primitive Data Types:

- Integer



## 2.2 Data Types in Python

Python has 4 Primitive Data Types:

- Integer
- Float





## 2.2 Data Types in Python

Python has 4 Primitive Data Types:

- Integer
- Float
- String



## 2.2 Data Types in Python

Python has 4 Primitive Data Types:

- Integer
- Float
- String
- Boolean



## 2.3 Data Types: Integer (int)

```
test7.py > ...
```

```
1 positiveNumber = 5  
2 negativeNumber = -3  
3 zeroNumber = 0  
4
```



## 2.3 Data Types: Integer (int)

```
test7.py > ...
```

```
1 positiveNumber = 5
2 negativeNumber = -3
3 zeroNumber = 0
4
```

- An **Integer** is a whole number that can hold negative numbers, positive numbers and zero.
- the type of these variables is determined by the value assigned to them, i.e. a whole number -> **Integer**



## 2.3 Data Types: Integer (int)

```
test7.py > ...
1 positiveNumber = 5
2 negativeNumber = -3
3 zeroNumber = 0
4
5 print('Positive Number: {}'.format(type(positiveNumber), positiveNumber))
6 print('Negative Number: {}'.format(type(negativeNumber), negativeNumber))
7 print('Zero Number: {}'.format(type(zeroNumber), zeroNumber))
8
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
maireadmeagher@mac dev % /opt/homebrew/bin/python3 /Users/maireadmeagher/Documents/GitRepos/Python-PF/dev/test7.py
TypeError: type() takes 1 or 3 arguments
maireadmeagher@mac dev % /opt/homebrew/bin/python3 /Users/maireadmeagher/Documents/GitRepos/Python-PF/dev/test7.py
Positive Number: <class 'int'> 5
Negative Number: <class 'int'> -3
Zero Number: <class 'int'> 0
```



## 2.3 Data Types: Integer (int)

```
test7.py > ...
1 positiveNumber = 5
2 negativeNumber = -3
3 zeroNumber = 0
4
5 print('Positive Number: {}'.format(type(positiveNumber), positiveNumber))
6 print('Negative Number: {}'.format(type(negativeNumber), negativeNumber))
7 print('Zero Number: {}'.format(type(zeroNumber), zeroNumber))
8
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
maireadmeagher@mac dev % /opt/homebrew/bin/python3 /Users/maireadmeagher/Documents/GitRepos/Python-PF/de
v/test7.py
TypeError: type() takes 1 or 3 arguments
maireadmeagher@mac dev % /opt/homebrew/bin/python3 /Users/maireadmeagher/Documents/GitRepos/Python-PF/de
v/test7.py
Positive Number: <class 'int'> 5
Negative Number: <class 'int'> -3
Zero Number: <class 'int'> 0
```

- We can check that the variables are **Integer (int)** by asking python to check the type of the variable using the built-in `type()` function.
- we will cover later what each part of this code is doing; you can ignore it for the moment!



## 2.4 Data Types: Float (float)

```
test8.py > ...
1 decimalNumber = 12.34
2 pi = 3.14159
3 celcius = - 34.4
4
5 print('Decimal Number: {}'.format(type(decimalNumber)), decimalNumber)
6 print('Pi: {}'.format(type(pi)), pi)
7 print('Celcius: {}'.format(type(celcius)), celcius)
```



## 2.4 Data Types: Float (float)

- An **Float** is a floating decimal point number that can hold negative numbers, positive numbers and zero.

```
test8.py > ...
1 decimalNumber = 12.34
2 pi = 3.14159
3 celcius = - 34.4
4
5 print('Decimal Number: {}'.format(type(decimalNumber)), decimalNumber)
6 print('Pi: {}'.format(type(pi)), pi)
7 print('Celcius: {}'.format(type(celcius)), celcius)
```





## 2.4 Data Types: Float (float)

- An **Float** is a floating decimal point number that can hold negative numbers, positive numbers and zero.
- We can check that the variables are **Float (float)** by asking python to check the type of the variable using the built-in `type()` function.

```
test8.py > ...
1 decimalNumber = 12.34
2 pi = 3.14159
3 celcius = - 34.4
4
5 print('Decimal Number: {}'.format(type(decimalNumber)), decimalNumber)
6 print('Pi: {}'.format(type(pi)), pi)
7 print('Celcius: {}'.format(type(celcius)), celcius)
```



## 2.4 Data Types: Float (float)

```
test8.py > ...
1 decimalNumber = 12.34
2 pi = 3.14159
3 celcius = - 34.4
4
5 print('Decimal Number: {}'.format(type(decimalNumber)), decimalNumber)
6 print('Pi: {}'.format(type(pi)), pi)
7 print('Celcius: {}'.format(type(celcius)), celcius)
```

- An **Float** is a floating decimal point number that can hold negative numbers, positive numbers and zero.
- We can check that the variables are **Float (float)** by asking python to check the type of the variable using the built-in `type()` function.
- As an exercise, check what the output of this code is.



## 2.5 Data Types: String (str)

```
test9.py > ...
1 firstString = "This is a string - it uses double quotes"
2 secondString = 'This is also a string - it uses single quotes'
3 Click to add a breakpoint String = 'Different characters - 4562~#*£$& '
4
5 print(firstString)
6 print(secondString)
7 print(differentCharsString)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
maireadmeagher@Mac dev % /opt/homebrew/bin/python3 /Users/maireadmeagher/Documents/GitRepos/Python-PF/dev/tes
This is a string - it uses double quotes
This is also a string - it uses single quotes
Different characters - 4562~#*£$&
```



## 2.5 Data Types: String (str)

```
test9.py > ...
1 firstString = "This is a string - it uses double quotes"
2 secondString = 'This is also a string - it uses single quotes'
3 Click to add a breakpoint String = 'Different characters - 4562~#*£$& '
4
5 print(firstString)
6 print(secondString)
7 print(differentCharsString)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
maireadmeagher@Mac dev % /opt/homebrew/bin/python3 /Users/maireadmeagher/Documents/GitRepos/Python-PF/dev/tes
This is a string - it uses double quotes
This is also a string - it uses single quotes
Different characters - 4562~#*£$&
maireadmeagher@Mac dev % ^
```

- A **String** is a sequence of characters that can hold text.



## 2.5 Data Types: String (str)

```
test9.py > ...
1 firstString = "This is a string - it uses double quotes"
2 secondString = 'This is also a string - it uses single quotes'
3 Click to add a breakpoint String = 'Different characters - 4562~#*£$& '
4
5 print(firstString)
6 print(secondString)
7 print(differentCharsString)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
maireadmeagher@Mac dev % /opt/homebrew/bin/python3 /Users/maireadmeagher/Documents/GitRepos/Python-PF/dev/tes
This is a string - it uses double quotes
This is also a string - it uses single quotes
Different characters - 4562~#*£$&
```

- A **String** is a sequence of characters that can hold text.
- They are created by enclosing characters in quotes (single or double).



## 2.6 Data Types: Boolean (bool)

```
test10.py > ...
1  isOpen = True
2  isClosed = False
3
4  print('Is the shop open? ' + str(isOpen))
5  print('Is the shop closed? ' + str(isClosed))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
maireadmeagher@Mac dev % /opt/homebrew/bin/python3 /Users/maireadmeagher/Docu
Is the shop open? True
Is the shop closed? False
maireadmeagher@Mac dev %
```



## 2.6 Data Types: Boolean (bool)

- A **Boolean** is a data type that can only have two values: True or False.

```
test10.py > ...
1  isOpen = True
2  isClosed = False
3
4  print('Is the shop open? ' + str(isOpen))
5  print('Is the shop closed? ' + str(isClosed))

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
maireadmeagher@Mac dev % /opt/homebrew/bin/python3 /Users/maireadmeagher/Docu
Is the shop open? True
Is the shop closed? False
maireadmeagher@Mac dev %
```



## 2.6 Data Types: Boolean (bool)



```
test10.py > ...
1  isOpen = True
2  isClosed = False
3
4  print('Is the shop open? ' + str(isOpen))
5  print('Is the shop closed? ' + str(isClosed))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
maireadmeagher@Mac dev % /opt/homebrew/bin/python3 /Users/maireadmeagher/Docu
Is the shop open? True
Is the shop closed? False
maireadmeagher@Mac dev %
```

- A **Boolean** is a data type that can only have two values: True or False.
- Booleans are often used in programming to represent the truth value of a condition or to control the flow of a program based on certain conditions.





## 2.7 Change a Variable's Data Type

- In Python, you can change the data type of a variable by assigning a new value of a different type to it. For example:



## 2.7 Change a Variable's Data Type

- In Python, you can change the data type of a variable by assigning a new value of a different type to it. For example:

```
num = 10                # num is an integer
print(type(num))        # Output: <class 'int'>
num = 3.14              # num is now a float
print(type(num))        # Output: <class 'float'>
num = "Hello"           # num is now a string
print(type(num))        # Output: <class 'str'>
```



## 2.8 Assigning multiple values to multiple variables

```
1 num1, num2, num3 = 1, 2, 3
2
3 print(num1)
4 print(num2)
5 print(num3)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL POP

```
maireadmeagher@Mac dev % /opt/homebrew/bin/python3 /U
1
2
3
```



## 2.8 Assigning multiple values to multiple variables

- Here we have defined three variables each with a starting value. We can change the value of each variable separately by assigning a new value to it.

```
1 num1, num2, num3 = 1, 2, 3
2
3 print(num1)
4 print(num2)
5 print(num3)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL POR

```
maireadmeagher@Mac dev % /opt/homebrew/bin/python3 /U
1
2
3
```



## 2.8 Assigning multiple values to multiple variables

- Here we have defined three variables each with a starting value. We can change the value of each variable separately by assigning a new value to it.
- The variables can be of different data types, and we can check the type of each variable using the `type()` function.

```
1 num1, num2, num3 = 1, 2, 3
2
3 print(num1)
4 print(num2)
5 print(num3)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL POR

```
maireadmeagher@Mac dev % /opt/homebrew/bin/python3 /U
1
2
3
```



## 2.8 Assigning multiple values to multiple variables

```
1 num1, num2, num3 = 1, 2, 3
2
3 print(num1)
4 print(num2)
5 print(num3)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL POR

```
maireadmeagher@Mac dev % /opt/homebrew/bin/python3 /U
1
2
3
```

- Here we have defined three variables each with a starting value. We can change the value of each variable separately by assigning a new value to it.
- The variables can be of different data types, and we can check the type of each variable using the `type()` function.
- As an exercise, print out the type of each variable, then change the value of one of the variables (using a different data type) and check using the `type()` function.



## 2.8 Assigning multiple values to multiple variables

```
test11.py / ...  
1 num1 = num2 = num3 = 1  
2  
3 print('num1 =', num1)  
4 print('num2 =', num2)  
5 print('num3 =', num3)  
  
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
maireadmeagher@Mac dev % /opt/homebrew/bin/python3 /Users/maire  
maireadmeagher@Mac dev % /opt/homebrew/bin/python3 /Users/maire  
num1 = 1  
num2 = 1  
num3 = 1  
maireadmeagher@Mac dev %
```



## 2.8 Assigning multiple values to multiple variables

```
test11.py / ...
1 num1 = num2 = num3 = 1
2
3 print('num1 =', num1)
4 print('num2 =', num2)
5 print('num3 =', num3)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
maireadmeagher@Mac dev % /opt/homebrew/bin/python3 /Users/maire
maireadmeagher@Mac dev % /opt/homebrew/bin/python3 /Users/maire
num1 = 1
num2 = 1
num3 = 1
maireadmeagher@Mac dev %
```

- Here we have defined three variables each with a starting value of 1.





# Outline

1. Variables

2. Data Types

**3. Arithmetic Operators**



### 3. Arithmetic Operators

- Arithmetic operators are symbols that perform mathematical operations on numbers.



### 3. Arithmetic Operators

- Arithmetic operators are symbols that perform mathematical operations on numbers.
- The basic arithmetic operators in Python are:

| Arithmetic Operator | Explanation    | Example(s)                            |
|---------------------|----------------|---------------------------------------|
| +                   | Addition       | $6 + 2$<br><code>diameter + 10</code> |
| -                   | Subtraction    | $6 - 2$<br><code>diameter - 10</code> |
| *                   | Multiplication | $6 * 2$<br><code>diameter * 10</code> |
| /                   | Division       | $6 / 2$<br><code>diameter / 10</code> |



## 3.1 Modulo Operator

- The modulo operator % returns the remainder of a division operation.



## 3.1 Modulo Operator

- The modulo operator `%` returns the remainder of a division operation.
- For example, `7 % 3` would return 1 because when you divide 7 by 3, the quotient is 2 with a remainder of 1.



## 3.1 Modulo Operator

- The modulo operator % returns the remainder of a division operation.
- For example,  $7 \% 3$  would return 1 because when you divide 7 by 3, the quotient is 2 with a remainder of 1.
- The modulo operator is often used to determine if a number is even or odd, or to cycle through a sequence of values.



## 3.2 Assignment with Arithmetic Operators

- In Python, you can use assignment operators to perform an arithmetic operation and assign the result back to the same variable. For example:



## 3.2 Assignment with Arithmetic Operators

```
num = 10
num = num + 5 # takes the current value of num (10), adds 5 to it, and
              # assigns the result (15) back to num
print(num)   # Output: 15
num = num - 3 # takes the current value of num (15), subtracts 3 from it,
              # and assigns the result (12) back to num
print(num)   # Output: 12
num = num * 2 # takes the current value of num (12), multiplies it by 2,
              # and assigns the result (24) back to num
print(num)   # Output: 24
num = num / 4 # takes the current value of num (24), divides it by 4, and
              # assigns the result (6.0) back to num
print(num)   # Output: 6.0
```





## 3.3 Shortcuts for Assignment with Arithmetic Operators

- Python provides shortcuts for assignment with arithmetic operators, which allow you to perform the operation and assignment in a single step. For example:



## 3.3 Shortcuts for Assignment with Arithmetic Operators

- Python provides shortcuts for assignment with arithmetic operators, which allow you to perform the operation and assignment in a single step. For example:

```
num = 10
num += 5 # equivalent to num = num + 5
print(num) # Output: 15
num -= 3 # equivalent to num = num - 3
print(num) # Output: 12
num *= 2 # equivalent to num = num * 2
print(num) # Output: 24
num /= 4 # equivalent to num = num / 4
print(num) # Output: 6.0
```



# Thanks for Watching - Any questions?

